
Tribhuvan University
Faculty of Humanities & Social Sciences
OFFICE OF THE DEAN

Microprocessor & Computer Architecture

BCA — Semester 2 — 2018 — Answer Sheet

Subject Code: CACS155

Group A

Attempt ALL the questions. [10 × 1 = 10]

-
1. 1. How many number of pins are there in 8085 Microprocessor?
a) 16 b) 20 c) 32 d) 40
 2. Which one of the following interrupt has the highest priority?
a) RST7.5 b) TRAP c) RST6.5 d) RST5.5
 3. How many bytes make a word of 32 bits?
a) One Byte b) Two Bytes c) Three Bytes d) Four Bytes
 4. Which one of the following flag has set or reset value on the basis of even or odd number of 1's in result?
a) Zero b) Carry c) Parity d) Sign
 5. What is the size of MOV B, A instruction in 8085 Microprocessor?
a) One Word b) Two Word c) Three Word d) Four Word
 6. Which one of the following bit(s) specify the direct or indirect address?
a) Address bits b) Opcode Bits c) Mode Bit d) Control Word
 7. Which one of the following term is correct for the process of transformation of the instruction code bits to an address in control memory where the routine is located?
a) Mapping b) Pipelining c) Sequencing d) Acknowledging
 8. Which one of the following is not a logical and bit manipulation operation?
a) Enable Interrupt b) Increment c) Clear Carry d) Clear
 9. Which one of the following is not the pipelining hazard?
a) Data dependency b) Resource conflict c) Branch conflict d) Interrupt Hazard
 10. Which one of the following organization of parallel processing is only a theoretical interest since no practical system has been constructed?
a) SISD b) SIMD c) MISD d) MIMD

[10 × 1 = 10]

Answer: Answers:

1) d) 40 — Intel 8085 is an 8-bit microprocessor with a 40-pin DIP (Dual Inline Package) configuration.

2) b) TRAP — TRAP is a non-maskable interrupt with the highest priority in 8085. It cannot be disabled and is used for critical conditions like power failure.

3) d) Four Bytes — 1 byte = 8 bits. Therefore, 32 bits = $32/8 = 4$ bytes.

4) c) Parity — The Parity flag is set if the result has an even number of 1s (even parity), and reset if odd number of 1s (odd parity).

5) a) One Word — MOV B, A is a 1-byte instruction that copies the contents of accumulator to register B.

6) c) Mode Bit — The mode bit specifies whether addressing is direct or indirect in microprogrammed control units.

7) a) Mapping — Mapping is the process that transforms instruction code bits into an address in control memory where the corresponding microprogram routine is stored.

8) a) Enable Interrupt — EI (Enable Interrupt) is a program control instruction, not a logical or bit manipulation operation. Logical operations include AND, OR, XOR, NOT, etc.

9) d) Interrupt Hazard — The three main pipeline hazards are: data hazards (data dependency), structural hazards (resource conflict), and control hazards (branch conflict). Interrupt hazard is not a standard pipeline hazard category.

10) c) MISD — MISD (Multiple Instruction, Single Data) is a theoretical parallel processing architecture where multiple instructions operate on the same data stream simultaneously. No practical MISD system has been constructed.

Group B

Attempt any SIX questions. [$6 \times 5 = 30$]

11. Explain the Bus organization of 8085 Microprocessor.

[5]

Answer: Bus Organization of 8085 Microprocessor:

The 8085 microprocessor uses three types of buses to communicate with memory and I/O devices:

1. Address Bus (16-bit):

- Unidirectional bus (CPU to memory/I/O)
- $A_{15}-A_1$ (high-order) and AD_7-AD_0 (low-order, multiplexed)
- Can address $2^{16} = 65,536$ (64KB) memory locations
- Used to specify the source or destination of data

2. Data Bus (8-bit):

- Bidirectional bus
- AD_7-AD_0 lines are multiplexed (shared with address bus low byte)
- ALE (Address Latch Enable) signal demultiplexes them
- Transfers 8 bits of data at a time between CPU and memory/I/O

3. Control Bus:

- Carries control and status signals
- **Control Signals:** RD (read), WR (write), IO/M (I/O or memory)
- **Status Signals:** S_1 , S_0 (identify operation type)
- **Other Signals:** ALE, READY, RESET IN, HOLD, HLDA

Bus Timing:

During T1 state: Address is placed on address bus, ALE goes high

During T2/T3 states: Data transfer occurs on data bus

This organized bus structure enables efficient communication between all system components.

12. Explain the opcode fetch machine cycle for MVI A, 32H with timing diagram.

[5]

Answer: Opcode Fetch Machine Cycle for MVI A, 32H:

MVI A, 32H is a 2-byte instruction:

- Byte 1: Opcode (3E H) — MVI A
- Byte 2: Data (32 H) — immediate operand

Opcode Fetch Cycle (T1-T4):

T1 State:

- PC contents (address of 3E H) placed on address bus
- ALE goes HIGH to latch the address
- IO/M $\square$ = 0 (memory operation)

T2 State:

- ALE goes LOW
- RD $\square$ goes LOW (read signal active)
- Memory places opcode 3E H on data bus

T3 State:

- Opcode 3E H is read into instruction register
- RD $\square$ goes HIGH
- PC is incremented by 1

T4 State:

- Opcode is decoded
- CPU recognizes it as MVI A (2-byte instruction)

Memory Read Cycle for Operand (T1-T3 of next cycle):

Similar to opcode fetch, but reads data byte 32H and transfers it to accumulator.

[Timing Diagram: CLK, ALE, A $\square$ $\square$ -A $\square$, AD $\square$ -AD $\square$, RD $\square$, WR $\square$ showing states T1-T4]

Total Machine Cycles: 2 (Opcode Fetch + Memory Read) = 7 T-states

13. Explain the 8085 Instruction addressing modes with example.

[5]

Answer: Addressing Modes of 8085:

1. Immediate Addressing:

Data is specified directly in the instruction.

Example: MVI A, 32H — loads 32H directly into accumulator

Example: ADI 05H — adds 05H to accumulator

2. Register Addressing:

Data is in a register specified in the instruction.

Example: MOV A, B — copies data from register B to A

Example: ADD C — adds contents of register C to A

3. Register Indirect Addressing:

Instruction specifies a register pair that holds the memory address of data.

Example: MOV A, M — loads data from memory location pointed by HL pair

Example: LDAX B — loads accumulator from memory pointed by BC pair

4. Direct Addressing:

Instruction contains the 16-bit memory address directly.

Example: LDA 2050H — loads accumulator from memory location 2050H

Example: STA 3050H — stores accumulator to memory location 3050H

5. Implicit/Implied Addressing:

No operand is specified; operation is internal to CPU.

Example: CMA — complements accumulator

Example: RLC — rotates accumulator left

Summary Table:

Mode	Data Location	Example
Immediate	In instruction	MVI A, 32H
Register	In register	MOV A, B
Register Indirect	Memory (addr in reg pair)	MOV A, M
Direct	Memory (addr in instruction)	LDA 2050H
Implicit/Internal	register	CMA

14. Explain the memory hierarchy with diagram.

[5]

Answer: Memory Hierarchy:

Memory hierarchy organizes computer memory into levels based on speed, cost, and size. The goal is to provide a balance between fast access and large capacity at reasonable cost.

[Diagram description: Pyramid structure]

Level	Type	Speed	Size	Cost
Top	Registers	Fastest (1 ns)	Smallest (bytes)	Highest
Cache (L1, L2, L3)		Very fast (2-10 ns)	Small (KB-MB)	Very high
Main Memory (RAM)		Fast (50-100 ns)	Medium (GB)	Moderate
Secondary (SSD/HDD)		Slow (ms)	Large (TB)	Low
Bottom	Offline/Tertiary (Tape)	Slowest	Largest	Lowest

Principle: As we go down the hierarchy:

- Speed decreases
- Capacity increases
- Cost per bit decreases

Working Principle:

- Most frequently used data is kept in faster, smaller memories (registers, cache)
- Less frequently used data resides in slower, larger memories (RAM, disk)
- Data moves between levels automatically (cache management, virtual memory)

Locality of Reference:

- **Temporal:** Recently accessed data is likely to be accessed again
- **Spatial:** Data near recently accessed data is likely to be accessed

This principle makes memory hierarchy effective, giving the illusion of large, fast, cheap memory.

15. Explain the organization of Microprogrammed Control Unit.

[5]

Answer: Microprogrammed Control Unit:

A microprogrammed control unit implements control signals using a stored program (microprogram) in control memory (ROM), rather than hardwired logic. It is more flexible and easier to modify than hardwired control.

Components:

1. Control Memory (ROM):

- Stores microinstructions (control words)
- Each machine instruction corresponds to a microprogram (sequence of microinstructions)
- Typically 1K-4K words of control store

2. Control Address Register (CAR):

- Holds the address of the next microinstruction to be executed
- Also called Microprogram Counter (μ PC)

3. Control Data Register (CDR):

- Holds the current microinstruction being executed
- Also called Pipeline Register or Control Buffer Register

4. Microinstruction Format:

Contains fields for:

- **Control Field:** Specifies which control signals to activate
- **Address Field:** Specifies address of next microinstruction
- **Condition Field:** Specifies branching conditions (flags, status)

5. Mapping Logic:

- Converts opcode of machine instruction to starting address of its microprogram in control memory

Execution Process:

1. Machine instruction fetched and decoded
2. Mapping logic converts opcode to microprogram address
3. Microinstructions are fetched from control memory sequentially
4. Control signals are generated to execute each microoperation
5. Next address logic determines next microinstruction address

Advantages: Flexible, easy to modify, supports complex instructions

Disadvantages: Slower than hardwired control, requires control memory

16. Define control word. Explain the procedure for generating control word for specific operation.

[5]

Answer: Control Word:

A control word is a binary word stored in control memory that specifies the control signals to be activated during a microoperation. Each bit in the control word corresponds to a specific control signal. When a bit is 1, the corresponding control signal is active; when 0, it is inactive.

Format of Control Word:

F1F2F3CDBRAD 3 bits3 bits3 bits2 bits2 bits7 bits

- **F1, F2, F3:** Microoperation fields (select ALU, register, or memory operations)
- **CD:** Condition for branching (status bits)
- **BR:** Branch field (type of branch: JMP, CALL, RET, etc.)
- **AD:** Address field (next microinstruction address)

Procedure for Generating Control Word:

Step 1: Identify the microoperations required for the specific operation (e.g., ADD, MOV, LOAD).

Step 2: Determine which control signals need to be activated for each microoperation.

Example for ADD R1, R2:

- Select R1 as destination $\rightarrow DR \leftarrow R1$
- Select R2 as source $\rightarrow SR \leftarrow R2$
- ALU performs ADD $\rightarrow ALU_op = ADD$
- Write result to R1 $\rightarrow Write_enable$

Step 3: Encode the control signals into the control word format.

Each control signal is assigned a bit position in the control word.

Step 4: Store the control word in control memory at the appropriate address.

Step 5: Sequence the control words to form a complete microprogram for the instruction.

Example Control Word for $PC \leftarrow PC + 1$:

- $PC_out = 1$ (output PC to bus)
- $ALU_add = 1$ (ALU adds 1)
- $PC_in = 1$ (input result to PC)
- All other signals = 0

Control Word = 0010 1000 0001... (binary pattern for active signals)

17. Define instruction pipeline. Explain the four-segment instruction pipeline with example.

[5]

Answer: Instruction Pipeline:

Instruction pipelining is a technique where multiple instructions are overlapped in execution. The processor is divided into stages, and different instructions occupy different stages simultaneously, similar to an assembly line. This increases instruction throughput (number of instructions completed per unit time).

Four-Segment Instruction Pipeline:

Segment 1: FI (Fetch Instruction)

- Read instruction from memory
- PC is incremented
- Instruction placed in instruction register

Segment 2: DA (Decode Address / Decode Instruction)

- Decode the opcode to determine the operation
- Decode/register fetch: read register operands
- Calculate effective address for memory operands

Segment 3: FO (Fetch Operands)

- Fetch operands from registers or memory
- For memory operands, perform address calculation and memory read

Segment 4: EX (Execute)

- ALU performs the operation
- Store result in destination register or memory
- Update status flags

Example Execution:

Consider instructions I1, I2, I3, I4:

Clock F I D A F O E X 1 I1 --- 2 I2 I1 -- 3 I3 I2 I1 - 4 I4 I3 I2 I1 5 I5 I4 I3 I2 6 I6 I5 I4 I3

After initial fill (3 cycles), one instruction completes every cycle. Without pipelining, each instruction would take 4 cycles, so 4 instructions = 16 cycles. With pipelining, 4 instructions = 7 cycles (after fill).

Group C

Attempt any TWO questions. [$2 \times 10 = 20$]

18. Explain the functional block diagram of 8085 Microprocessor.

[10]

Answer: Functional Block Diagram of 8085 Microprocessor:

The 8085 is an 8-bit NMOS microprocessor with the following functional units:

[Diagram description: Central ALU connected to registers, control unit, and buses]

1. Arithmetic and Logic Unit (ALU):

- Performs arithmetic operations (ADD, SUB, INC, DEC)
- Performs logical operations (AND, OR, XOR, NOT, Compare)
- Updates flags (Sign, Zero, Carry, Parity, Auxiliary Carry)
- 8-bit wide, works with accumulator and temporary register

2. Registers:

- **Accumulator (A):** 8-bit, stores result of ALU operations
- **General Purpose Registers:** B, C, D, E, H, L (8-bit each)
- **Register Pairs:** BC, DE, HL (16-bit each for addressing)
- **Program Counter (PC):** 16-bit, holds address of next instruction
- **Stack Pointer (SP):** 16-bit, points to top of stack
- **Instruction Register (IR):** Holds current instruction
- **Temporary Registers:** W, Z, ACT, TMP for internal operations

3. Flag Register (8-bit):

- S (Sign): Set if MSB of result is 1
- Z (Zero): Set if result is zero
- AC (Auxiliary Carry): Carry from bit 3 to bit 4
- P (Parity): Set if result has even number of 1s
- CY (Carry): Set if carry/borrow generated

4. Timing and Control Unit:

- Generates control signals (RD $\square$, WR $\square$, ALE, IO/ $\square$)
- Manages instruction sequencing and machine cycles
- Synchronizes operations with clock (3 MHz)

5. Interrupt Control:

- Manages 5 interrupts: INTR, RST 5.5, 6.5, 7.5, TRAP
- TRAP is non-maskable with highest priority

6. Serial I/O Control:

- SID (Serial Input Data) and SOD (Serial Output Data)
- Enables serial communication

7. Address Buffer and Address/Data Buffer:

- Address buffer drives 16-bit address bus
- Address/Data buffer is bidirectional for multiplexed AD $\square$ -AD $\square$

All these units work together to fetch, decode, and execute instructions in a coordinated manner.

19. Explain the design and control logic of Accumulator.

[10]

Answer: Accumulator in 8085:

The accumulator (register A) is an 8-bit register that plays a central role in 8085 operations. It is the default source and destination for most ALU operations.

Design of Accumulator:

1. 8-bit Register:

- Built using 8 D flip-flops
- Can store one byte of data
- Connected to internal data bus through tri-state buffers

2. Input Path:

- Data can be loaded into accumulator from:
 - Internal data bus (from memory, I/O, or other registers)
 - ALU output (result of arithmetic/logical operations)
- Load control signal: A_in (when active, data enters accumulator)

3. Output Path:

- Accumulator contents can be output to:
 - Internal data bus (to memory, I/O, or other registers)
 - ALU input A (for ALU operations)
- Output control signal: A_out (when active, data leaves accumulator)

Control Logic of Accumulator:

1. Load Operation ($A \leftarrow \text{data}$):

- Control signals: A_in = 1, A_out = 0
- Data from bus is latched into accumulator on clock edge
- Used by instructions: MVI A, data; MOV A, reg; LDA addr; IN port

2. Output Operation ($\text{bus} \leftarrow A$):

- Control signals: A_out = 1, A_in = 0
- Accumulator contents placed on internal bus
- Used by instructions: MOV reg, A; STA addr; OUT port

3. ALU Operation ($A \leftarrow A \text{ op data}$):

- Control signals: A_out = 1 (to ALU), A_in = 1 (from ALU)
- Both operations may occur in different clock phases
- Used by: ADD, SUB, ANA, ORA, XRA, CMP instructions

4. Special Operations:

- **Complement:** $A \leftarrow A \oplus \text{FF}$ (CMA instruction)
- **Decimal Adjust:** $A \leftarrow \text{BCD adjust}$ (DAA instruction)
- **Rotate:** RLC, RRC, RAL, RAR operations

Timing:

In T-state operations, accumulator input is enabled during write cycles and output during read cycles, synchronized by the internal clock and control unit signals.

Importance:

The accumulator is essential because:

- Most arithmetic/logical operations use it implicitly
- Flag register is updated based on accumulator contents
- I/O operations transfer data through accumulator

20. Explain the different data transfer and manipulation instructions with example.

[10]

Answer: Data Transfer and Manipulation Instructions in 8085:

A. DATA TRANSFER INSTRUCTIONS:

1. MOV (Move):

Transfers data between registers or between register and memory.

Example: MOV A, B — copies content of B to A

Example: MOV M, A — copies A to memory location pointed by HL

2. MVI (Move Immediate):

Loads 8-bit immediate data into register or memory.

Example: MVI A, 32H — loads 32H into accumulator

Example: MVI B, 05H — loads 05H into register B

3. LDA (Load Accumulator Direct):

Loads accumulator from direct 16-bit memory address.

Example: LDA 2050H — $A \leftarrow [2050H]$

4. STA (Store Accumulator Direct):

Stores accumulator to direct 16-bit memory address.

Example: STA 3050H — $[3050H] \leftarrow A$

5. LHLD (Load HL Direct):

Loads HL register pair from two consecutive memory locations.

Example: LHLD 2050H — $L \leftarrow [2050H], H \leftarrow [2051H]$

6. SHLD (Store HL Direct):

Stores HL register pair to two consecutive memory locations.

Example: SHLD 3050H — $[3050H] \leftarrow L, [3051H] \leftarrow H$

7. LXI (Load Extended Immediate):

Loads 16-bit immediate data into register pair.

Example: LXI H, 2050H — $HL \leftarrow 2050H$

8. IN/OUT (Input/Output):

Transfers data between accumulator and I/O port.

Example: IN 01H — $A \leftarrow \text{data from port 01H}$

Example: OUT 02H — $\text{port 02H} \leftarrow A$

B. DATA MANIPULATION INSTRUCTIONS:

1. Arithmetic Instructions:

• **ADD:** ADD B — $A \leftarrow A + B$

• **ADI:** ADI 05H — $A \leftarrow A + 05H$

• **SUB:** SUB C — $A \leftarrow A - C$

• **SUI:** SUI 03H — $A \leftarrow A - 03H$

• **INR:** INR B — $B \leftarrow B + 1$

-
- **DCR:** $\text{DCR } C \rightarrow C \leftarrow C - 1$
 - **DAD:** $\text{DAD } B \rightarrow \text{HL} \leftarrow \text{HL} + \text{BC}$

2. Logical Instructions:

- **ANA:** $\text{ANA } B \rightarrow A \leftarrow A \text{ AND } B$ (bitwise AND)
- **ANI:** $\text{ANI } 0\text{FH} \rightarrow A \leftarrow A \text{ AND } 0\text{FH}$ (mask upper nibble)
- **ORA:** $\text{ORA } C \rightarrow A \leftarrow A \text{ OR } C$ (bitwise OR)
- **XRA:** $\text{XRA } A \rightarrow A \leftarrow A \text{ XOR } A = 0$ (clear accumulator)
- **CMA:** $\text{CMA} \rightarrow A \leftarrow A$ (complement accumulator)
- **CMP:** $\text{CMP } B \rightarrow$ Compare A with B ($A - B$, flags updated)

3. Rotate Instructions:

- **RLC:** Rotate accumulator left (MSB $\rightarrow$ LSB and Carry)
- **RRC:** Rotate accumulator right (LSB $\rightarrow$ MSB and Carry)
- **RAL:** Rotate left through carry
- **RAR:** Rotate right through carry

Summary: Data transfer instructions move data without modification, while manipulation instructions perform arithmetic, logical, and shift operations that transform data.

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.